GCP Setup for Cluster Logging and Network Observability

OpenShift on GCP with Workload Identity Federation (WIF) · hub `local-cluster` (ocp4.lab.internal) · OpenShift 4.22 · 8 November 2026

1. Purpose and current status

Two policy groups in the GitOps repository send data to Google Cloud: **GCP logging** (application, infrastructure and audit logs to Cloud Logging) and **Network Observability** (network flows stored by Loki in a Cloud Storage bucket). On a WIF cluster they authenticate with short-lived tokens, so no service account keys are stored on the cluster.

The Cloud Credential Operator (CCO) runs in Manual mode. It turns CredentialsRequests into credential Secrets, but it cannot create GCP service accounts, IAM bindings or buckets. Those must be created once per cluster, which is what this guide covers.

Status on 2026-11-08. Logging: policy Compliant, but the collector fails with "Failed to get GCP access token" and drops logs. The service account it impersonates, `openshift-logging-writer@openenv-fs9jx.iam.gserviceaccount.com`, was not created for the rebuilt cluster. Network Observability: not enabled (cluster not labelled).

2. Cluster values

Item	Value	Source
GCP project ID	`openenv-fs9jx`	`infrastructure/cluster` status
GCP project number	`611605993704`	WIF audience
Region	`us-central1`	`infrastructure/cluster` status
Infrastructure name	`ocp4-l49rg`	`infrastructure/cluster` status
Workload identity pool / provider	`ocp4-lab` / `ocp4-lab`	WIF audience
Loki bucket	`openenv-fs9jx-ocp4-l49rg-loki`	`<project ID>-<infra name>-loki`
ManagedCluster name	`local-cluster`	ACM hub

3. What gets created

	Cluster Logging	Network Observability (Loki)
GCP role	`roles/logging.logWriter`	`roles/storage.objectAdmin`
Kubernetes ServiceAccounts	`openshift-logging/logcollector`	`netobserv/loki`, `netobserv/loki-ruler`
CredentialsRequest	`openshift-logging-gcp-wif` (created by the policy)	Written by hand for ccoctl; the Loki Operator creates its own once the LokiStack exists
Secret written by the CCO	`openshift-logging/gcp-wif-secret`	`netobserv/loki-managed-credentials`
Other GCP object	—	Cloud Storage bucket

4. Before you start

- **Linux host** (for example the bastion). `ccoctl` is shipped for Linux only.
- `oc` logged in to the cluster as cluster-admin, `gcloud` and `jq` installed.
- GCP identity with **Service Account Admin**, **Project IAM Admin** and **Storage Admin** on `openenv-fs9jx`.
- For the ccoctl procedure: the repository change that lets the policies read the generated service account emails (pull request #18, `docs/gcp-wif-ccoctl.md`) must be merged to `main`. Without it, use Appendix A (plain gcloud with fixed names).

5. Procedure with ccoctl

`ccoctl` names each service account `<name, 12 chars>-<CredentialsRequest name, 11 chars>-<5 random chars>`, so the email is not known in advance. After creating the accounts you hand their emails to the policies through the hub ConfigMap `rhacm-policies/gcp-wif-service-accounts`.

Step 1 – Set variables

```
PROJECT_ID=$(oc get infrastructure cluster -o jsonpath='{.status.platformStatus.gcp.projectID}')
REGION=$(oc get infrastructure cluster -o jsonpath='{.status.platformStatus.gcp.region}')
INFRA=$(oc get infrastructure cluster -o jsonpath='{.status.infrastructureName}')
AUDIENCE=$(oc get secret cloud-credentials -n openshift-ingress-operator \
  -o jsonpath='{.data.service_account\.json}' | base64 -d | jq -r .audience)
POOL=$(echo "$AUDIENCE" | cut -d/ -f9)
PROVIDER=$(echo "$AUDIENCE" | cut -d/ -f11)
CLUSTER=local-cluster
echo "$PROJECT_ID $REGION $INFRA $POOL $PROVIDER"
# expected: openenv-fs9jx us-central1 ocp4-l49rg ocp4-lab ocp4-lab
```

Step 2 – Extract ccoctl from the cluster release image

```
RELEASE_IMAGE=$(oc get clusterversion version -o jsonpath='{.status.desired.image}')
oc get secret pull-secret -n openshift-config \
  -o jsonpath='{.data.\.dockerconfigjson}' | base64 -d > pull-secret.json
oc adm release extract --registry-config pull-secret.json \
  --command=ccoctl --to=./bin "$RELEASE_IMAGE"
chmod +x bin/ccoctl && rm pull-secret.json
```

Step 3 – Log in to GCP

`ccoctl` uses Application Default Credentials.

```
gcloud auth application-default login
gcloud config set project "$PROJECT_ID"
```

Step 4 – Collect the CredentialsRequests

```
mkdir -p credreqs

# Logging: created on the cluster by the day2-logging-gcp-forwarder policy
oc get credentialsrequest openshift-logging-gcp-wif \
  -n openshift-cloud-credential-operator -o yaml > credreqs/logging.yaml

# Loki: read by ccoctl only, never applied to the cluster
cat > credreqs/loki.yaml <<'EOF'
apiVersion: cloudcredential.openshift.io/v1
kind: CredentialsRequest
metadata:
  name: loki-netobserv
  namespace: openshift-cloud-credential-operator
spec:
  serviceAccountNames:
    - loki
    - loki-ruler
  secretRef:
    name: loki-managed-credentials
    namespace: netobserv
  providerSpec:
    apiVersion: cloudcredential.openshift.io/v1
    kind: GCPProviderSpec
    predefinedRoles:
      - roles/storage.objectAdmin
    skipServiceCheck: true
EOF
```

`ccoctl` grants `predefinedRoles` on the whole project, so the Loki account can use every bucket in it. For a bucket-only grant, delete the `predefinedRoles` lines and use the optional binding in Step 6.

Step 5 – Create the service accounts

> Use `create-service-accounts`, **not** `create-all`. The workload identity pool, provider and OIDC issuer were created at install time and must not be replaced.

```
./bin/ccoctl gcp create-service-accounts \
  --name="$INFRA" \
  --project="$PROJECT_ID" \
  --workload-identity-pool="$POOL" \
  --workload-identity-provider="$PROVIDER" \
  --credentials-requests-dir=credreqs \
  --output-dir=ccoctl-out
```

For each CredentialsRequest this creates the GCP service account, grants its role on the project and grants `roles/iam.workloadIdentityUser` to each Kubernetes ServiceAccount through the pool. Read the generated emails:

```
gsa() {
  grep 'service_account.json' "$1" | awk '{print $2}' | base64 -d \
    | jq -r .service_account_impersonation_url \
    | sed -e 's#.*/serviceAccounts/##' -e 's#:generateAccessToken##'
}
LOGGING_GSA=$(gsa ccoctl-out/manifests/openshift-logging-gcp-wif-secret-credentials.yaml)
LOKI_GSA=$(gsa ccoctl-out/manifests/netobserv-loki-managed-credentials-credentials.yaml)
echo "$LOGGING_GSA"; echo "$LOKI_GSA"
```

> Do **not** apply the Secret manifests in `ccoctl-out/manifests`. The CCO writes the Secrets from the CredentialsRequests, and the logging collector reads its token from `/var/run/ocp-collector/serviceaccount/token`, not the path ccoctl puts in its manifests.

Step 6 – Create the Loki bucket

`ccoctl` does not create buckets. The name must match what the policy derives.

```
BUCKET=${PROJECT_ID}-${INFRA}-loki
gcloud storage buckets create "gs://${BUCKET}" --project "$PROJECT_ID" \
  --location "$REGION" --uniform-bucket-level-access

# Only if predefinedRoles was removed in Step 4:
# gcloud storage buckets add-iam-policy-binding "gs://${BUCKET}" \
#   --member "serviceAccount:${LOKI_GSA}" --role roles/storage.objectAdmin
```

Step 7 – Hand the emails to the policies (on the hub)

```
oc create configmap gcp-wif-service-accounts -n rhacm-policies \
  --from-literal="${CLUSTER}.logging=${LOGGING_GSA}" \
  --from-literal="${CLUSTER}.loki=${LOKI_GSA}" \
  --dry-run=client -o yaml | oc apply -f -
```

The ConfigMap is not in Git, so Argo CD leaves it alone. ACM re-renders both policies, and the CCO rewrites `openshift-logging/gcp-wif-secret` with the new service account.

Step 8 – Restart the log collector

The collector caches a failed token exchange, so restart it once the Secret shows the new email.

```
oc get secret gcp-wif-secret -n openshift-logging \
  -o jsonpath='{.data.service_account\.json}' | base64 -d \
  | jq -r .service_account_impersonation_url
oc delete pod -n openshift-logging -l app.kubernetes.io/component=collector
```

Step 9 – Enable Network Observability

```
oc label managedcluster "$CLUSTER" network-observability=enabled
```

6. Verify

```
# Logging: expect 0, then check Logs Explorer for the cluster's log ID
oc logs -n openshift-logging ds/gcp-logging -c collector --since=10m \
  | grep -c 'Failed to get GCP access token'

# Network Observability
oc get lokistack loki -n netobserv
oc get secret loki-managed-credentials -n netobserv
oc get flowcollector cluster -o jsonpath='{.status.conditions[?(@.type=="Ready")]}'
oc logs -n netobserv loki-ingester-0 --since=10m | grep -i 'failed to flush'   # empty = OK
gcloud storage ls "gs://${BUCKET}/" | head
```

7. Troubleshooting

Symptom	Cause	Fix
Collector: "Failed to get GCP access token", events dropped	Service account missing, wrong email, or cached failure	Steps 5, 7 and 8; check the email in `gcp-wif-secret`
"Gaia id not found for email ..."	The GCP service account does not exist	Create it (Step 5 or Appendix A)
403 / PERMISSION_DENIED on the bucket	Loki account lacks `roles/storage.objectAdmin`	Grant the role on the project or bucket
Policy Compliant but no data in GCP	Policies check objects, not delivery	Use the checks in section 6
`day2-netobserv-*` policies missing	Cluster not labelled	Step 9

Appendix A – Without ccoctl (gcloud, fixed names)

Works with the current `main` branch. Without the hub ConfigMap the policies expect `openshift-logging-writer@<project ID>` and `openshift-loki@<project ID>`. Set the variables from Step 1 first.

Cluster Logging

```
POOL_PATH=projects/611605993704/locations/global/workloadIdentityPools/${POOL}
SUBJ=principal://iam.googleapis.com/${POOL_PATH}/subject/system:serviceaccount
GSA=openshift-logging-writer@${PROJECT_ID}.iam.gserviceaccount.com
gcloud iam service-accounts create openshift-logging-writer --project "$PROJECT_ID"
gcloud projects add-iam-policy-binding "$PROJECT_ID" \
  --member "serviceAccount:${GSA}" --role roles/logging.logWriter
gcloud iam service-accounts add-iam-policy-binding "$GSA" --project "$PROJECT_ID" \
  --role roles/iam.workloadIdentityUser \
  --member "${SUBJ}:openshift-logging:logcollector"
oc delete pod -n openshift-logging -l app.kubernetes.io/component=collector
```

Network Observability

```
POOL_PATH=projects/611605993704/locations/global/workloadIdentityPools/${POOL}
SUBJ=principal://iam.googleapis.com/${POOL_PATH}/subject/system:serviceaccount
BUCKET=${PROJECT_ID}-${INFRA}-loki
GSA=openshift-loki@${PROJECT_ID}.iam.gserviceaccount.com
gcloud storage buckets create "gs://${BUCKET}" --project "$PROJECT_ID" \
  --location "$REGION" --uniform-bucket-level-access
gcloud iam service-accounts create openshift-loki --project "$PROJECT_ID"
gcloud storage buckets add-iam-policy-binding "gs://${BUCKET}" \
  --member "serviceAccount:${GSA}" --role roles/storage.objectAdmin
for KSA in loki loki-ruler; do
  gcloud iam service-accounts add-iam-policy-binding "$GSA" --project "$PROJECT_ID" \
    --role roles/iam.workloadIdentityUser \
    --member "${SUBJ}:netobserv:${KSA}"
done
oc label managedcluster local-cluster network-observability=enabled
```

Appendix B – Removing

> Do not run `ccoctl gcp delete` for this. It is meant for cluster teardown and also removes the workload identity pool and OIDC bucket the cluster itself needs.

```
gcloud iam service-accounts delete "$LOGGING_GSA" --project "$PROJECT_ID"
gcloud iam service-accounts delete "$LOKI_GSA" --project "$PROJECT_ID"
oc delete configmap gcp-wif-service-accounts -n rhacm-policies
oc label managedcluster local-cluster network-observability-
```

Delete the Loki bucket only if its flow data is no longer needed: `gcloud storage rm -r "gs://${BUCKET}"`.